St Francis Institute of Technology

(Autonomous Institute)

Department of Artificial Intelligence and Machine Learning

Third Year AIML Engineering (SEM-V A.Y. 2026–27)

Deep Learning Lab. Experiment Report

Expt.-11: SDG-Based Deep Learning Mini Project

Project title IndicDoc AI: Where Do Layout Detectors Fail on Indian Documents? A Study of Script, Domain and Scan Quality

Team

Name	Roll No.
Riddhi Patil	39
Mariam Badure	40
Prashant Jha	41
Aditya Soni	42

Project guide Dr. Joanne Gomes

SDGs addressed SDG 4 (Quality Education), SDG 9 (Industry, Innovation and Infrastructure), SDG 10 (Reduced Inequalities), SDG 16 (Peace, Justice and Strong Institutions)

Code repository `github.com/va-ada/IndicDoc-AI`

Date of report 24 October 2026

Aim

To design, implement, and analyze a Deep Learning solution addressing a real-world problem aligned with the Sustainable Development Goals (SDGs).

Lab Outcome

Students will be able to apply Deep Learning techniques for real world problem-solving and evaluate model performance.

Methodology

The project is carried out in multiple phases to ensure systematic development and implementation:

1. **Topic Selection (SDG Mapping):** Identify a real-world problem aligned with Sustainable Development Goals (SDGs). *Done:* layout understanding and text extraction for Indian-language documents (Section 1).

2. **Phase-1: Literature Survey:** Review 4–8 research papers and identify the research gap. *Done:* eight papers reviewed, three gaps identified (Sections 2–3).

3. **Phase-2: Final Project Implementation with Performance Analysis:** Develop system architecture, collect dataset, and implement DL model. Analyze results, improve performance, design front end, integrate the system. Demonstrate the finished project and submit the code along with final working project. *Done:* Sections 4–16.

4. **Documentation:** Prepare a research paper in standard format. *Done:* IEEE conference-format paper in `paper/`.

***Abstract*—**Indian offices, schools and archives hold millions of scanned pages in a dozen scripts. Before such a page can be searched, read aloud or converted for a blind reader, a system must find its parts (paragraphs, headings, tables, figures, page numbers) and then read the text in the right order. We fine-tuned two convolutional object detectors, Faster R-CNN and YOLOv8s, on a language-balanced 2,293-page subset of the IndicDLP dataset [1] and compared them with the DocLayout-YOLO model released by the dataset's authors on the same 682 held-out test pages covering 12 languages and 42 region labels. The released model reaches 0.759 mAP@50 (0.575 mAP@50:95); our YOLOv8s reaches 0.259 and our Faster R-CNN 0.148 after about one hour of training each on a laptop GPU. We add two simple measures, a Consistency Index and a Robustness Index, and find that accuracy is fairly even across languages (CI 0.86) but not across document types (CI 0.55), and that the best model keeps 97% of its accuracy on damaged scans. A region-based script probe picks the right OCR language on 106/120 pages (Tesseract's own detector: 70/120). The work is delivered as IndicDoc AI, a web app and REST API in one offline Docker image, with accessible HTML and EPUB3 export for screen-reader users.

***Index Terms*—**document layout analysis, Indian languages, object detection, transfer learning, robustness, OCR, accessibility.

1. Topic Selection and SDG Mapping

Problem. Most document-AI tools are built and tested on English. Indian documents use many scripts (Devanagari, Bengali, Tamil, Telugu, Kannada, Malayalam, Gujarati, Gurmukhi, Odia, Latin) and dense formats (multi-column newspapers, question papers with nested sub-questions, legal acts with numbered clauses). If the layout step fails, OCR reads columns straight across and every later step fails with it. This hits hardest the people who depend on machines to read for them: blind and low-vision students, who study from state-board textbooks that are mostly available only as scans, and citizens who need government notices in their own language. The mapping to the SDG targets of the 2030 Agenda [20] is given in Table I.

TABLE I
MAPPING OF THE PROJECT TO THE SUSTAINABLE DEVELOPMENT GOALS

SDG	**Target**	**How IndicDoc AI contributes**
SDG 4 Quality Education	4.5: equal access to education for persons with disabilities	Turns scanned textbook and exam pages into structured, screen-reader-ready HTML / EPUB3 in their own language
SDG 9 Industry, Innovation & Infrastructure	9.5: enhance research and technological capability	Open, offline, reproducible document-AI pipeline for 12 Indian languages; measured study of its weak spots

SDG	Target	How IndicDoc AI contributes
SDG 10 Reduced Inequalities	10.2: social inclusion irrespective of disability, ethnicity or origin	Serves speakers of 11 Indian languages besides English, which commercial tools cover unevenly
SDG 16 Strong Institutions	16.10: public access to information	Makes scanned government circulars, acts and forms searchable and readable

2. Literature Survey Summary Table

Eight recent papers on Indian-language document understanding were reviewed in Phase-1 (Table II). They cover layout datasets [1], [3], text recognition [2], [7], table understanding [4], [8] and multimodal document models [5], [6].

TABLE II
LITERATURE SURVEY SUMMARY

Ref	Author	Problem	Dataset	Method	Limitation
[1]	Nath *et al.*, 2025	Layout parsing of Indian-language pages	IndicDLP: 119,806 pages, 12 languages, 12 domains, 42 labels	YOLOv10x, DocLayout-YOLO, RoDLA, DINO, DiT, Florence-2	Vision-only; 20–25 mAP drop on unseen scripts; no robustness test
[2]	Kasuba *et al.*, 2025	Page-level handwritten Indic text recognition	CHIPS, ~25K pages, 10 languages	DBNet word detection + 6 recognisers (CRNN, PARSeq, ...)	Synthetic pages; user must name the language
[3]	Mondal *et al.*, 2026	Multilingual table detection	UniTabBank: 55K pages, 82K tables, 28 languages	Semi-automatic labelling; YOLOv11 detector	Borderless tables and degraded scans fail
[4]	Pal *et al.*, 2024	Table QA for low-resource Indic languages	BanglaTabQA 2M, HindiTabQA 643K	SQL-generated QA; fine-tuned mBART / M2M / Llama	Wikipedia tables only; small test sets
[5]	Xu *et al.*, 2021	Multilingual document understanding	30M documents; XFUND, 7 languages	LayoutXLM: text + layout + image transformer	No Indian language; needs good OCR
[6]	Kasuba *et al.*, 2025	Grounding answers on the page	MGVG: 70 circulars, 509 QA pairs	OCR + Llama-3.1-8B + region matching	OCR-dependent; single page; small test set
[7]	Bhattacharyya *et al.*, 2025	Hindi OCR with vision-language models	1.1M printed Hindi words + pages	LoRA-tuned GOT-OCR 2.0, progressive learning	Hindi only; mixed-language lines
[8]	Dumada & Shah, 2026	Devanagari table extraction	500+ Devanagari documents	Structure + Tesseract content, weighted fusion	Self-reported results; no ablation

3. Research Gaps

1. **Only clean pages are tested.** IndicDLP [1] benchmarks large detectors on clean pages; no study measures how accuracy holds up on blurred, noisy, compressed, low-resolution or faded scans, which are the norm in real archives and phone photos (addressed by RQ4).
2. **Results are one overall number.** Papers report a single mAP. It is not known whether accuracy is even across the 12 languages and 12 document types, or which labels fail and why (RQ2, RQ3).
3. **OCR tools expect the user to name the language** [2], [7]. A multilingual tool needs to choose the script automatically, page by page (RQ5).

4. **No accessible output.** None of the surveyed systems produces structured, screen-reader-ready output (headings, lists, tables, language tag) from a scanned Indian-language page.

Position. Strong detectors already exist [1], [9]. We therefore do not claim a new detector: our contribution is a careful study of where they fail on Indian documents, and a working, accessible tool built on the best one.

4. Problem Statement and Objectives

Problem statement. Given an image or PDF page in any of 12 Indian languages, detect and classify every structural region into one of the 42 IndicDLP labels, read the text of each region in reading order, and measure, in simple numbers, where deep-learning layout detectors work and where they fail.

Main objective. Build and evaluate a deep-learning layout-and-text pipeline for Indian-language documents and deliver it as a usable tool.

Specific objectives.

1. Fine-tune two detectors (Faster R-CNN, YOLOv8s) with transfer learning and compare them with the published IndicDLP model on the same held-out test pages.
2. Define two simple measures, a Consistency Index and a Robustness Index, and use them to answer five research questions (RQ1–RQ5, Table III).
3. Add OCR with automatic script detection for 12 languages, and accessible HTML / EPUB3 export.
4. Ship a web app and REST API in one Docker image that works offline.

TABLE III
Research questions

RQ	Question
RQ1	How far behind is a detector trained on a small budget, compared with one trained on the full dataset?
RQ2	Is accuracy even across the 12 languages and the 12 document domains?
RQ3	Which region types are hardest, and is that because they are rare in the training data?
RQ4	How much accuracy survives blur, noise, JPEG compression, low resolution and faded print?
RQ5	Can the detected regions be used to choose the right OCR language automatically?

Expected output. A labelled box with a confidence score for every region; the page text in reading order as Markdown, JSON and accessible HTML / EPUB3; and the measured study results.

5. Proposed System, Methodology and Novelty

Workflow (input to output).

1. **Input:** a page image (PNG, JPG, TIFF) or a PDF page, rendered at 150 dpi.

2. **Detect regions:** a convolutional object detector (DocLayout-YOLO by default) outputs a box, one of 42 labels and a confidence for every region.
3. **Clean up:** boxes below the chosen confidence are dropped; boxes covering the same region with different labels (IoU > 0.7) are merged.
4. **Pick the script:** one detected text region is read with each of the 12 Tesseract script models; the model that returns the most characters of its own script wins.
5. **Read text:** Tesseract 5 [13] reads the page once; each word is assigned to the region that contains it, so words outside every region are kept rather than lost.
6. **Order:** regions are sorted into columns and reading order.
7. **Output:** annotated image, Markdown / JSON, and accessible HTML / EPUB3 (semantic headings, lists, tables, `lang` attribute).

How deep learning is used. Layout detection is posed as object detection. Convolutional detectors pre-trained on COCO [12] are fine-tuned on IndicDLP pages (transfer learning): the backbone learns visual features of text blocks, rules and figures, the neck combines them across scales, and the head predicts boxes and labels. The released DocLayout-YOLO [9], pre-trained on synthetic documents and fine-tuned on the full IndicDLP training set, is used as the production model.

Novelty. (i) *Study:* two interpretable ratios reported next to mAP — the Consistency Index $\mathrm{CI} = \min_{\mathrm{g}} \mathrm{mAP}_{50}(\mathrm{g}) \,/\, \max_{\mathrm{g}} \mathrm{mAP}_{50}(\mathrm{g})$ over languages or domains, and the Robustness Index $\mathrm{RI}_{\mathrm{c}} = \mathrm{mAP}_{50}(\text{damaged by c}) \,/\, \mathrm{mAP}_{50}(\text{clean})$ — and, to our knowledge, the first scan-damage robustness test on IndicDLP. (ii) *Tool:* two operating modes whose thresholds are chosen on validation pages ("find every part" and "balanced"); automatic script choice by probing detected regions; accessible EPUB3 output from scanned pages; one offline app for 12 languages.

6. System Block Diagram

Fig. 1 shows the major components and the flow of data. The same Python modules serve the web app, the REST API and the evaluation scripts, so the numbers in this report are produced by the code the user runs.

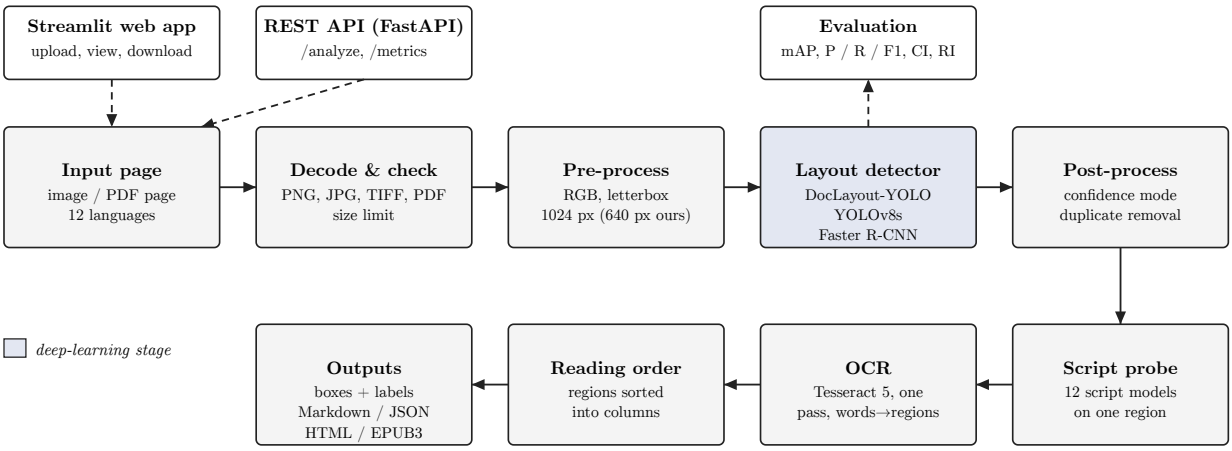


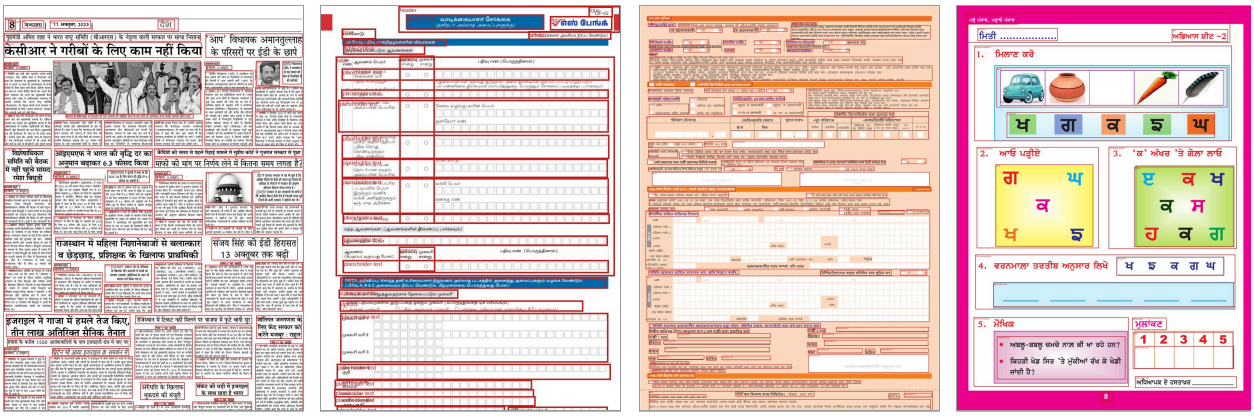
Fig. 1. System block diagram of IndicDoc AI. Solid arrows: data flow for one page; dashed arrows: requests from the two interfaces and the evaluation path. The deep-learning stage is shaded.

7. Dataset Description and Pre-processing

Dataset. IndicDLP [1], [18] (AI4Bharat, IIT Madras and IIIT Hyderabad; ICDAR 2025; also listed on India's AIKosh platform): 119,806 human-annotated pages, 12 languages, 12 document domains, 42 region labels. The full dataset is about 91 GB, so for our own two models we built a language-balanced subset of 3,317 pages and 49,995 boxes, taken from the *official* train / validation / test splits so that no test page was seen in training (Table IV). The subset is released as CSV files in `dataset/` (pages, boxes, class counts, language split) with one sample page per language (Fig. 2).

TABLE IV
IndicDLP subset used in this project (pages per split)

Language	Train	Validation	Test	Total
Assamese	129	30	60	219
Bengali	200	30	60	290
English	188	30	60	278
Gujarati	200	30	60	290
Hindi	200	21	22	243
Kannada	200	21	60	281
Malayalam	200	30	60	290
Marathi	200	30	60	290
Odia	200	30	60	290
Punjabi	188	30	60	278
Tamil	200	30	60	290
Telugu	188	30	60	278
Total pages	**2,293**	**342**	**682**	**3,317**
Labelled boxes	33,667	5,815	10,513	49,995



hindi · newspapers *tamil · forms* *bengali · forms* *punjabi · brochures*

Fig. 2. Sample pages from the dataset (one per language is shipped in `dataset/samples/`).

TABLE V
PRE-PROCESSING AND AUGMENTATION

Step	**What was done**
Loading	Only the needed Parquet row groups are streamed from Hugging Face (fsspec); images saved as PNG, annotations as COCO JSON
Cleaning	Boxes clipped to the page; boxes smaller than 1 px dropped (0.5% of training boxes)
Format conversion	COCO JSON → YOLO text labels for YOLOv8s
Resizing	Letterbox to 640 px (our models) or 1024 px (DocLayout-YOLO); Faster R-CNN short side 640–1024
Augmentation (YOLOv8s)	Mosaic, HSV colour jitter, scale and translate; horizontal flips off (a mirrored page is not a realistic document)
Augmentation (Faster R-CNN)	Colour jitter and mild blur; no flips
Inference input	PDF pages rendered at 150 dpi; RGB conversion; optional deskew and CLAHE contrast enhancement

Challenges in the data. Heavy class imbalance: *paragraph* alone is 32% of all boxes and 10 of the 42 labels have fewer than 100 training boxes (Fig. 3); two labels have almost no examples in the subset. Hindi has only 22 test pages in the official split, so its numbers are less certain. IndicDLP provides no text ground truth, so OCR character accuracy cannot be measured on it.

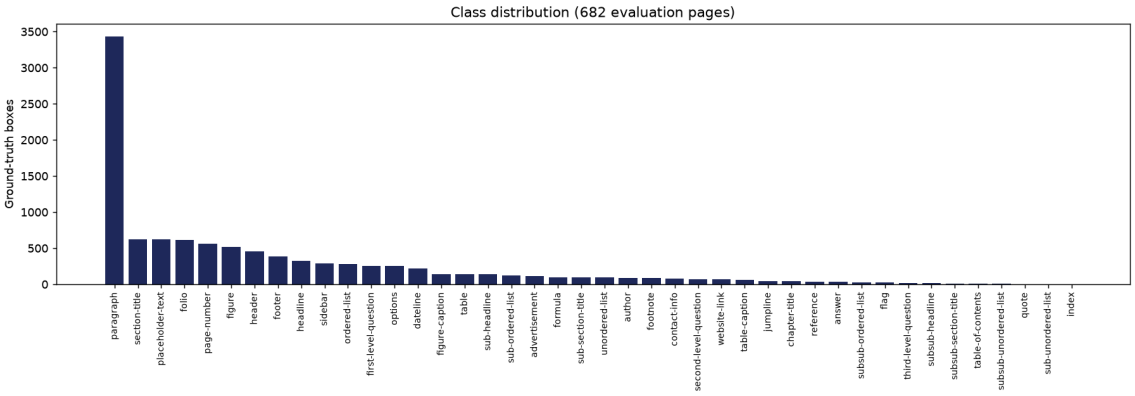


Fig. 3. Number of ground-truth boxes per label in the 682 test pages: a long-tailed distribution dominated by *paragraph*.

8. Deep Learning Model Architecture

Three detectors are compared (Fig. 4, Table VI). All follow the backbone–neck–head structure of modern object detectors.

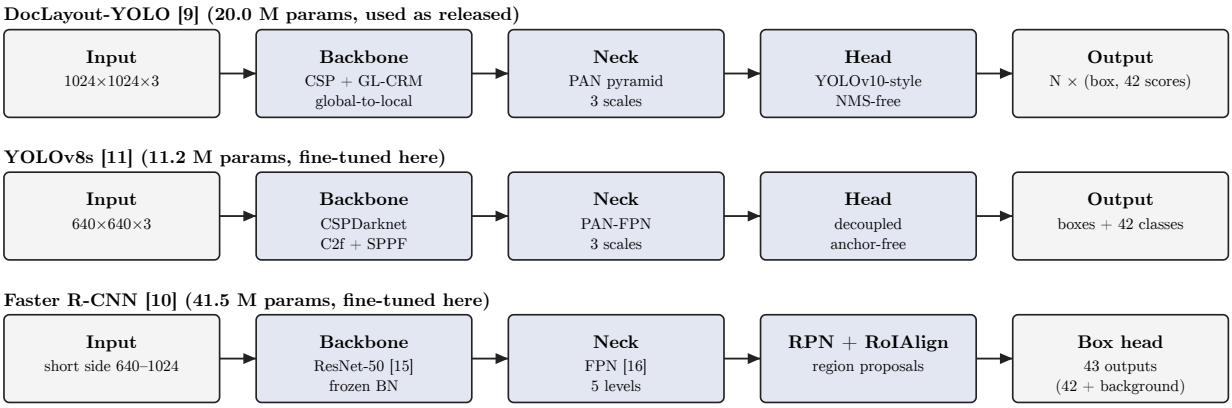


Fig. 4. Architectures of the three detectors with input and output dimensions. Shaded blocks are the learned feature extractor and prediction layers.

TABLE VI
Model details

	DocLayout-YOLO	**YOLOv8s**	**Faster R-CNN**
Role	production model	proposed (ours)	baseline (ours)
Type	one-stage, NMS-free	one-stage, anchor-free	two-stage
Parameters	20.0 M	11.2 M	41.5 M
Model size	161.5 MB	22.5 MB	166.6 MB
Input	$1024 \times 1024 \times 3$	$640 \times 640 \times 3$	short side 640–1024
Output	N × (box, 42 scores)	N × (box, 42 scores)	N × (box, 43 scores)
Pre-training	DocSynth-300K, then full IndicDLP by its authors [19]	COCO [12]	COCO [12]
Modification	none (used as released)	80-class head replaced by 42-class head	predictor replaced (FastRCNNPredictor, 43 classes)
Loss	—	CIoU + DFL (box) + BCE (class)	RPN + cross-entropy + smooth-L1

Major components. *Backbone* (CSPDarknet / ResNet-50 [15]) extracts visual features of strokes, text lines and rules at several resolutions. *Neck* (PAN / FPN [16]) fuses fine and coarse features so that both a folio number and a full-page table can be detected. *Head* predicts the box coordinates and a score for each of the 42 labels; Faster R-CNN first proposes candidate regions (RPN) and then classifies each RoIAligned region. DocLayout-YOLO adds a global-to-local controllable receptive module (GL-CRM) suited to document elements whose sizes vary greatly [9].

9. Tools, Technologies and Environment

TABLE VII
Tools, technologies and environment

Component	**Details**
Programming language	Python 3.11 (ML, app, API); JavaScript only for an experimental React prototype
DL framework	PyTorch 2 [17] with torchvision; Ultralytics 8 (YOLOv8) [11]; doclayout-yolo 0.0.4
Development environment	VS Code and terminal, Git + GitHub, pytest
Libraries	pycocotools (COCO mAP), OpenCV, Pillow, NumPy, pandas, Matplotlib, Plotly, PyMuPDF, Tesseract 5 (tessdata_best, 12 scripts)
Hardware	Apple M1 Pro laptop, 16-core GPU via PyTorch MPS (training and evaluation); CPU inside Docker
Dataset tools	Hugging Face Hub + fsspec (partial Parquet reads); custom COCO / YOLO converters
Interface / deployment	Streamlit web app + FastAPI REST API; Docker + docker-compose (works offline)

10. Implementation Details

The repository is organised as reusable modules (`ml/`) with thin entry points (`app/`, `backend/`, `scripts/`).

TABLE VIII
IMPLEMENTATION SUMMARY

Part	Implementation (file)
Data loading	Language-balanced sampling from the official splits; COCO JSON + YOLO labels (`scripts/prepare_indicdlp.py`, `ml/datasets/`)
Model construction	COCO-pretrained YOLOv8s and Faster R-CNN with new 42-class heads; released DocLayout-YOLO loaded as is (`ml/models/`)
Loss / optimiser	Faster R-CNN: SGD, lr 0.005, momentum 0.9, weight decay 5×10^{-4}, cosine schedule. YOLOv8s: Ultralytics automatic optimiser, CIoU + DFL + BCE
Batch / epochs	Faster R-CNN batch 4, 2 epochs (76-min budget); YOLOv8s batch 8, 12 epochs (60-min budget); seed 42
Validation	Best checkpoint chosen on 342 validation pages; operating thresholds chosen on validation, never on test (`ml/training/`)
Inference	One page at a time, duplicates removed, two modes (`ml/inference/predict.py`)
OCR and output	Script probe, word-to-region assignment, reading order, Markdown / JSON (`ml/ocr/`); accessible HTML / EPUB3 (`ml/accessible.py`, `scripts/pdf_to_epub.py`)
Interface	Streamlit app (`app/streamlit_app.py`), FastAPI (`backend/main.py`), Dockerfile + docker-compose
Quality	143 automated tests passed in the last recorded run (141 Python, 2 front-end); fixes logged in `docs/FINAL_AUDIT.md`

Listing 1 shows the core inference path used by the app, the API and the evaluation scripts.

Listing 1. One page in, labelled boxes and text out.

```
# ml/inference/predict.py - layout detection
model = load_model(ckpt, "doclayout", device)          # DocLayout-YOLO, YOLOv8s or Faster R-CNN
out = predict_single(model, page, "doclayout", device, conf=0.15)
out = dedupe_boxes(out, iou_threshold=0.7)             # merge same-region boxes

# ml/ocr - read the text inside the regions
doc = extract_document(page, out, languages="auto")    # script probe + Tesseract
print(doc["markdown"])                                 # text in reading order

# ml/accessible.py - screen-reader-ready output
html = to_html(doc, title="Chapter 1")                 # headings, lists, tables, lang tag
```

Listing 2. YOLOv8s fine-tuning call (ml/training/train_yolo.py, abridged).

```
model = get_proposed_model(model_size="s")             # COCO-pretrained YOLOv8s
model.train(data=yaml_path, imgsz=640, batch=8,
            seed=SEED, fliplr=0.0,                     # no mirror flips
            time=args.hours)                           # schedule fitted to a 1-hour budget
```

REST API. `POST /analyze` accepts an image or PDF and returns the boxes, labels and scores as JSON; `?ocr=true` adds the text with the script detected automatically. `GET /metrics` returns the saved evaluation results.

11. Experimental Setup and Training

TABLE IX
EXPERIMENTAL SETUP

Item	Setting
Data sizes	train 2,293 · validation 342 · test 682 pages (test = 10,513 boxes)
Faster R-CNN	ResNet-50 FPN, 640–1024 px, batch 4, SGD 0.005, 2 epochs, 76 min
YOLOv8s	640 px, batch 8, mosaic on, no flips, 12 epochs, 60 min
DocLayout-YOLO	1024 px, released checkpoint, evaluated only
Hardware	Apple M1 Pro GPU (MPS); timing measured one page at a time
Metrics	COCO mAP@50, mAP@50:95, mAP@75 (pycocotools, up to 300 boxes per page); precision, recall, F1 at a fixed confidence; average IoU; seconds per page
E1 (RQ1–RQ3)	All three detectors on the 682 clean test pages, broken down by language, domain and label
E2 (RQ4)	DocLayout-YOLO and YOLOv8s on the same pages under five kinds of scan damage (Table XII)
E3 (RQ5)	OCR script choice on 120 test pages (10 per language): region probe vs. Tesseract OSD

Fig. 5 shows the measured training curves. Both losses fall steadily and the validation loss follows the training loss, so neither model over-fits within its budget; the YOLOv8s validation mAP@50 is still rising at epoch 12 (0.309), which shows that the budget, not the architecture, limits our models. Within the one-hour budget, the one-stage YOLOv8s completed 12 epochs while the two-stage Faster R-CNN completed 2.

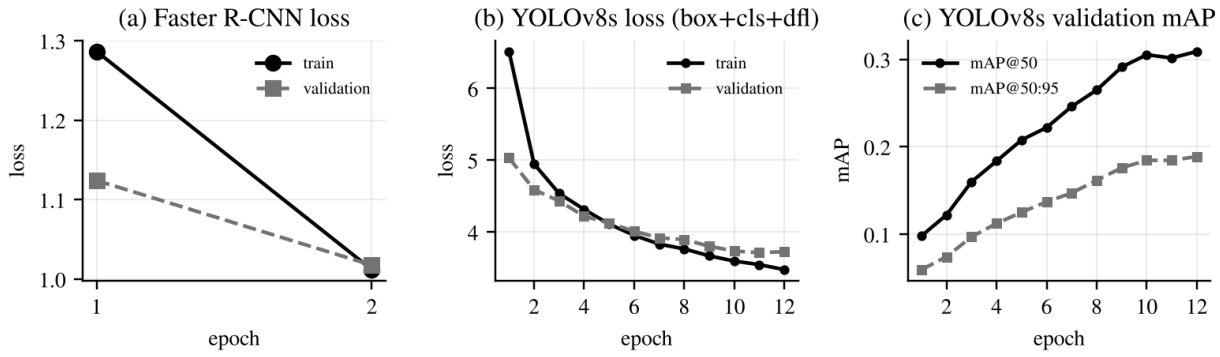


Fig. 5. Training curves (measured): (a) Faster R-CNN training and validation loss; (b) YOLOv8s loss; (c) YOLOv8s validation mAP per epoch.

12. Results and Performance Evaluation

All results are measured by `scripts/evaluate.py` on the same 682 held-out test pages (10,513 ground-truth boxes) that none of the models was trained on (Table X). Precision, recall and F1 are reported at the "balanced" confidence threshold chosen on validation pages for each model.

TABLE X
OVERALL RESULTS ON 682 TEST PAGES

Metric	DocLayout-YOLO	YOLOv8s (ours)	Faster R-CNN (ours)
mAP@50	**0.759**	0.259	0.148
mAP@50:95	**0.575**	0.165	0.079
mAP@75	**0.633**	0.172	0.075
Balanced threshold (from validation)	0.35	0.25	0.45
Precision	**0.848**	0.603	0.510
Recall	**0.775**	0.508	0.435
F1	**0.810**	0.551	0.469
Recall, "find every part" mode	**0.851**	0.508	0.435
Average IoU of correct boxes	**0.899**	0.885	0.821
Inference time (s / page, M1 Pro GPU)	0.150	**0.024**	0.202

RQ1 (budget gap). The released model, trained on about 42× more pages, is far ahead (0.759 vs. 0.259 mAP@50). In "find every part" mode it finds 85% of all regions at 74% precision. YOLOv8s is the fastest model (0.024 s per page) and 7× smaller than Faster R-CNN.

TABLE XI
MAP@50 PER LANGUAGE AND LANGUAGE CONSISTENCY INDEX

Language	Pages	DocLayout-YOLO	YOLOv8s	Faster R-CNN
Assamese	60	0.835	0.390	0.273
Bengali	60	0.757	0.303	0.199
English	60	0.768	0.315	0.186
Gujarati	60	0.779	0.308	0.190
Hindi	22	0.879	0.368	0.230
Kannada	60	0.757	0.355	0.186
Malayalam	60	0.776	0.432	0.270
Marathi	60	0.829	0.383	0.246
Odia	60	0.793	0.339	0.234
Punjabi	60	0.822	0.329	0.165
Tamil	60	0.807	0.339	0.223
Telugu	60	0.814	0.357	0.195
Consistency Index (CI)		**0.86**	0.70	0.61

RQ2 (languages and domains). Accuracy is fairly even across languages for the full-data model (CI 0.86; Table XI) and less even for our smaller models (0.70, 0.61). Across document domains it is much less even: CI 0.55 for DocLayout-YOLO, from 0.889 on research papers to 0.485 on forms (Fig. 6). *The type of document matters more than the script* once the script has been seen in training.

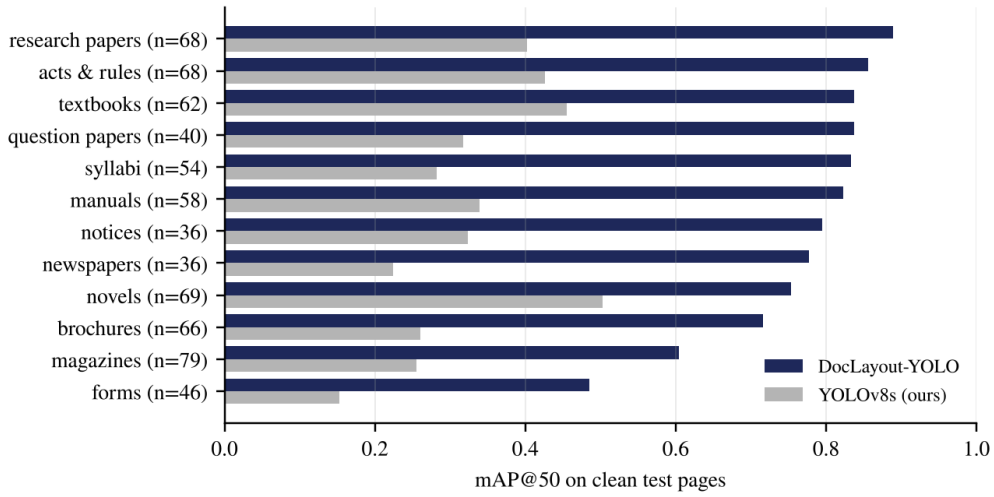


Fig. 6. mAP@50 per document domain (n = number of test pages). Forms and magazines are the hardest domains for both models.

RQ3 (rare labels). For DocLayout-YOLO the easiest labels are *paragraph* (0.95), *advertisement* (0.95), *ordered-list* (0.91), *figure* (0.90) AP@50 and the hardest are *quote* (0.06), *sub-unordered-list* (0.19), *sub-section-title* (0.47), *subsub-headline* (0.48). The Spearman rank correlation between a label's training frequency and its AP is 0.75 for our YOLOv8s but only 0.40 for the full-data model (Fig. 7): with enough data, rarity stops deciding accuracy, and what remains hard are labels that look alike (quote vs. paragraph, nested list levels).

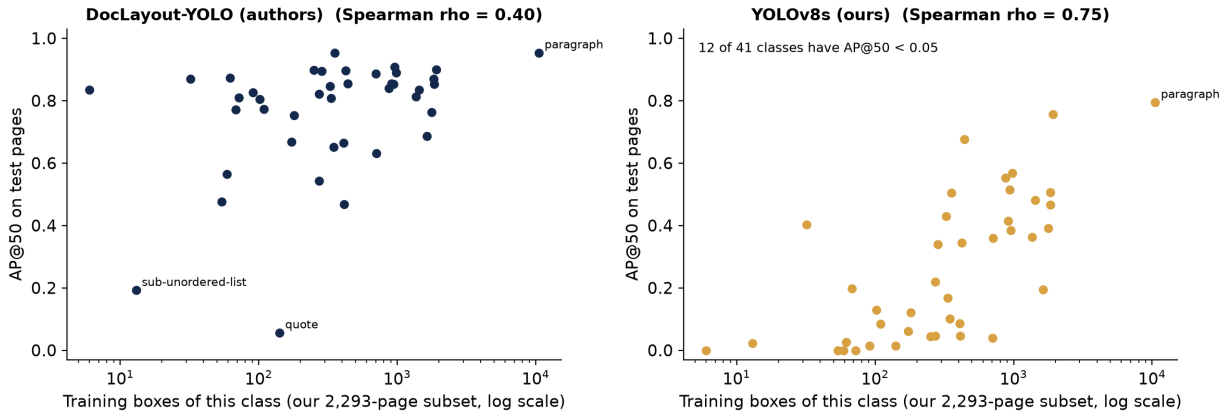


Fig. 7. AP@50 of each label against its number of training boxes (log scale). Rare labels are hard for the small-data model ($\rho = 0.75$), much less so for the full-data model ($\rho = 0.40$).

TABLE XII
ROBUSTNESS TO SCAN DAMAGE (E2): MAP@50 AND ROBUSTNESS INDEX

Condition	How it is made	DocLayout-YOLO	RI	YOLOv8s	RI
Clean	—	0.759	1.00	0.259	1.00
Blur	Gaussian blur, radius 2 px (out of focus)	0.704	0.93	0.242	0.94
Noise	Gaussian noise, $\sigma = 20$ grey levels	0.754	0.99	0.230	0.89
JPEG	JPEG quality 15 (forwarded phone image)	0.746	0.98	0.255	0.98
Half resolution	down to 50% and back up (low-dpi scan)	0.743	0.98	0.260	1.01
Faded print	contrast $\times$ 0.45, brightness $\times$ 1.25	0.732	0.97	0.226	0.87
Mean RI			**0.97**		**0.94**

RQ4 (damaged scans). The released model keeps 97% of its accuracy on average and YOLOv8s 94% (Table XII, Fig. 8). Blur hurts the released model most (RI 0.93); noise and faded print hurt the small model most (RI 0.89 and 0.87). Each damaged copy is made once with a fixed seed and the ground-truth boxes are unchanged.

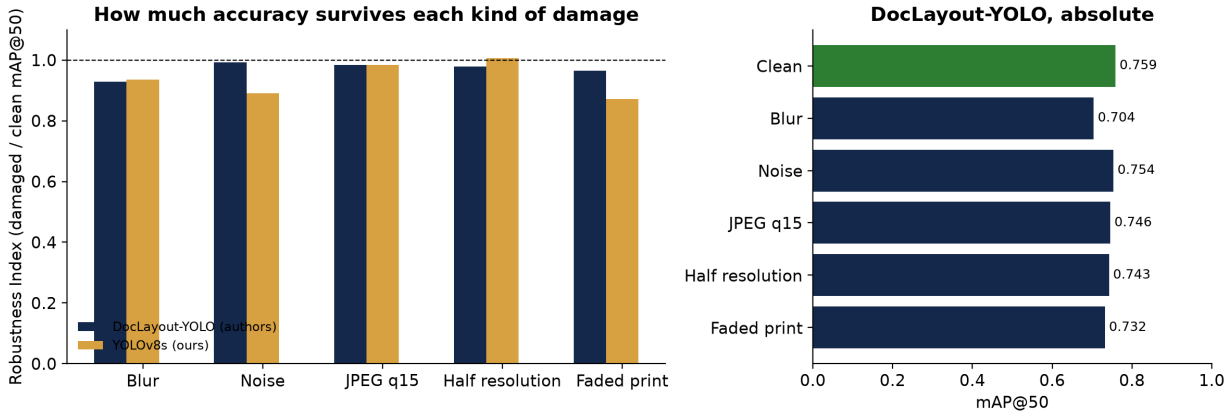


Fig. 8. Robustness Index per damage type (left) and absolute mAP@50 of DocLayout-YOLO under each condition (right).

RQ5 (choosing the OCR script). The region probe picks the right script on 106 of 120 pages (88%) against 70/120 for Tesseract's built-in detector (OSD). It is perfect on Assamese, Bengali and Punjabi and weakest on Marathi (6/10), where pages are sometimes mistaken for Latin or Telugu (Fig. 9).

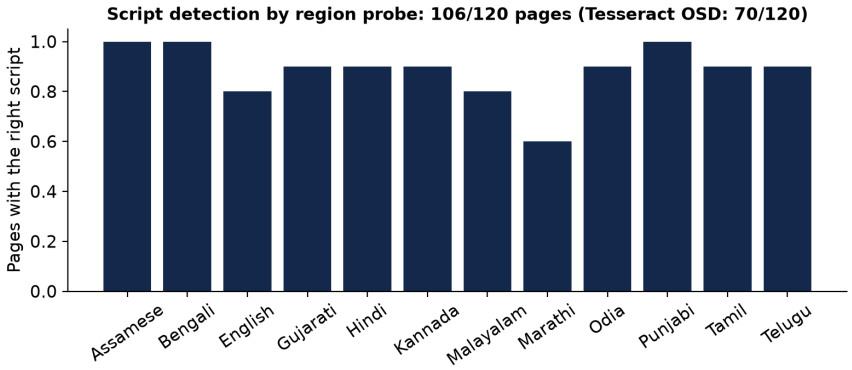


Fig. 9. Share of pages per language on which the region probe chose the right OCR script (106/120 overall).

13. Comparison with Baseline / Existing Methods

TABLE XIII
Comparison with the baseline and published results

Method	**Training data**	**mAP@50**	**mAP@50:95**
Faster R-CNN (ours, baseline)	2,293 pages, 76 min	0.148	0.079
YOLOv8s (ours, proposed)	2,293 pages, 60 min	0.259	0.165
DocLayout-YOLO (released by [1])	~96K pages (80% of 119,806)	**0.759**	**0.575**
Published: DocLayout-YOLO [1]	full IndicDLP test set	≈ 0.735	0.545
Published: YOLOv10x [1] (best in paper)	full IndicDLP test set	—	0.550

Metrics used. COCO mAP@50 and mAP@50:95, precision / recall / F1 and seconds per page, all on the same 682 test pages (rows above the line); published numbers are on the authors' full test set and are shown for reference. Our measurement of the released model (0.575 mAP@50:95) agrees with the published 0.545, which confirms that our evaluation pipeline is correct.

One-stage vs. two-stage. With the same data and a similar time budget, YOLOv8s beats Faster R-CNN on every metric (0.259 vs. 0.148 mAP@50; precision 0.603 vs. 0.510) and is 9× faster with a quarter of the parameters. The proposed model therefore improves on the baseline.

Did we beat the state of the art? No. Our own models saw about 42× less data and one hour of laptop training, and do not reach the published model. The application therefore uses the released DocLayout-YOLO, and the contribution of this project is the study (RQ1–RQ5), the OCR and script-selection pipeline, and the accessible, offline tool. We report this openly in the paper as well.

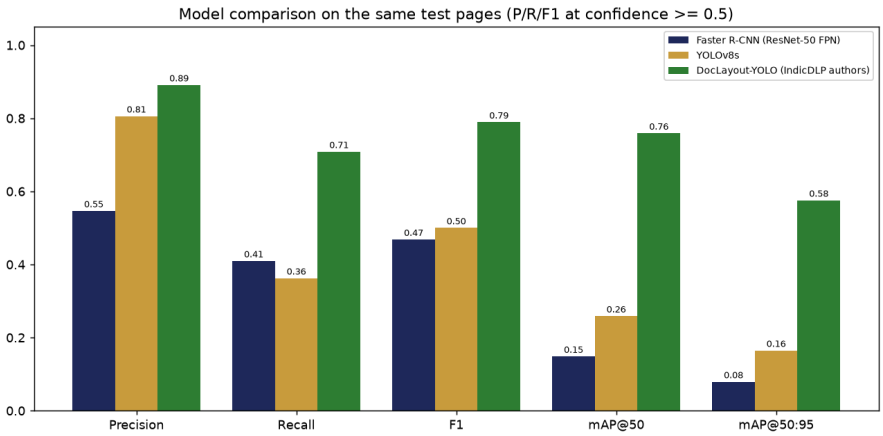


Fig. 10. Precision, recall, F1 (confidence ≥ 0.5) and mAP of the three models on the same test pages.

14. Sample Outputs and Demonstration

Figs. 11–15 give evidence that the implementation works end to end: the running application, a correct and an incorrect prediction, and the confusion matrix.



Fig. 11. Running Streamlit application. Left: a Tamil textbook page with the detected regions and confidences. Right: OCR with the script detected automatically (Tamil), text in reading order, Markdown / JSON download.

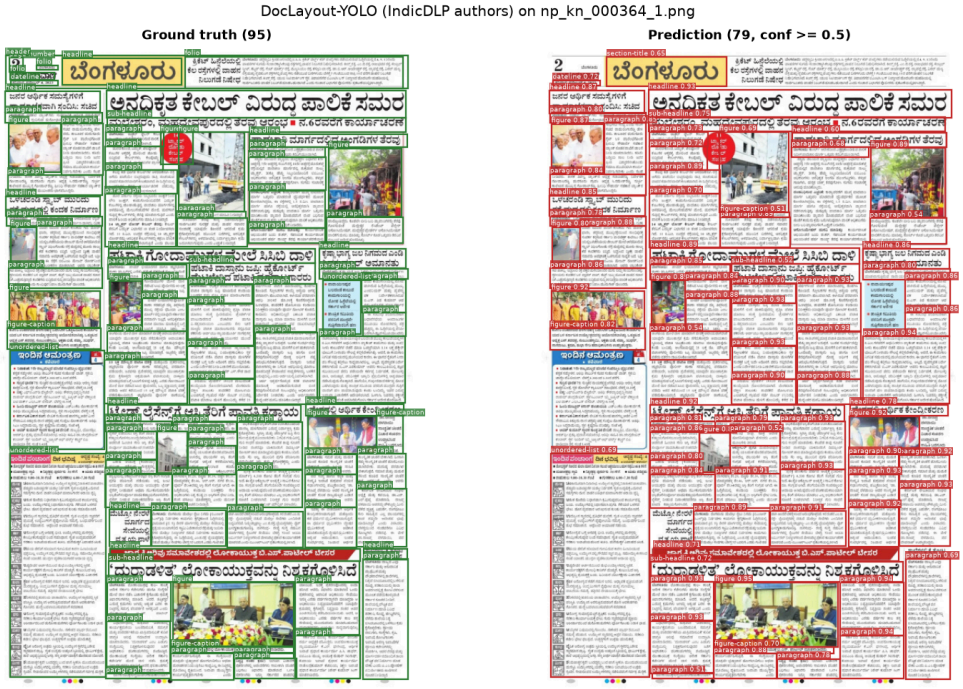


Fig. 12. Correct prediction: Kannada newspaper page, ground truth (green, left) vs. DocLayout-YOLO prediction (red, right).

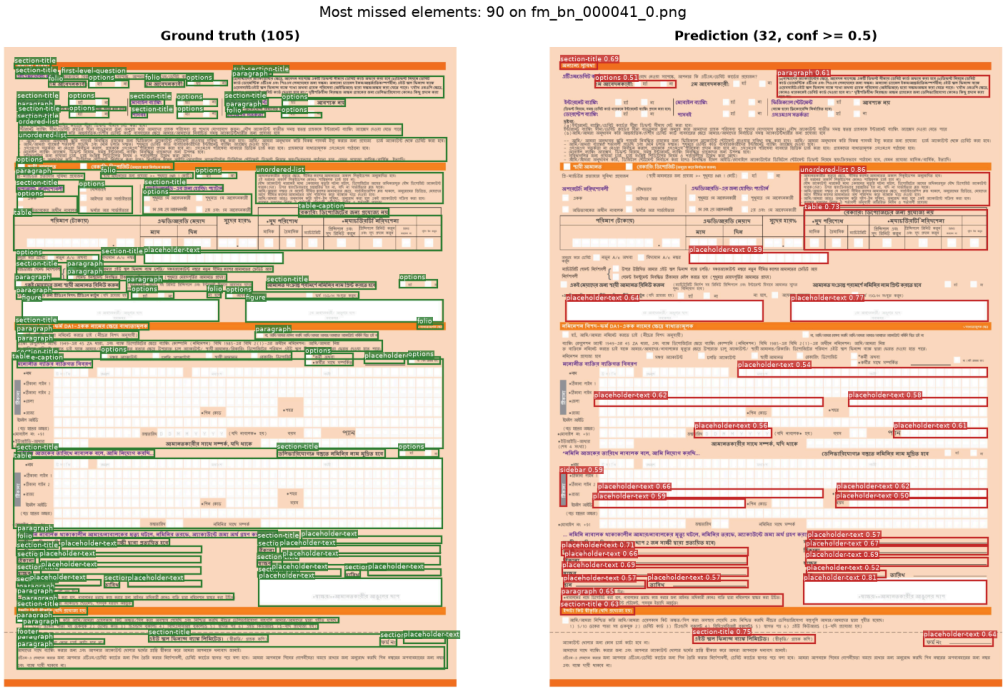


Fig. 13. Incorrect prediction: the dense Bengali form with the most missed regions in the test set (90 of 105 regions missed). Forms with many tiny fields are the hardest domain.

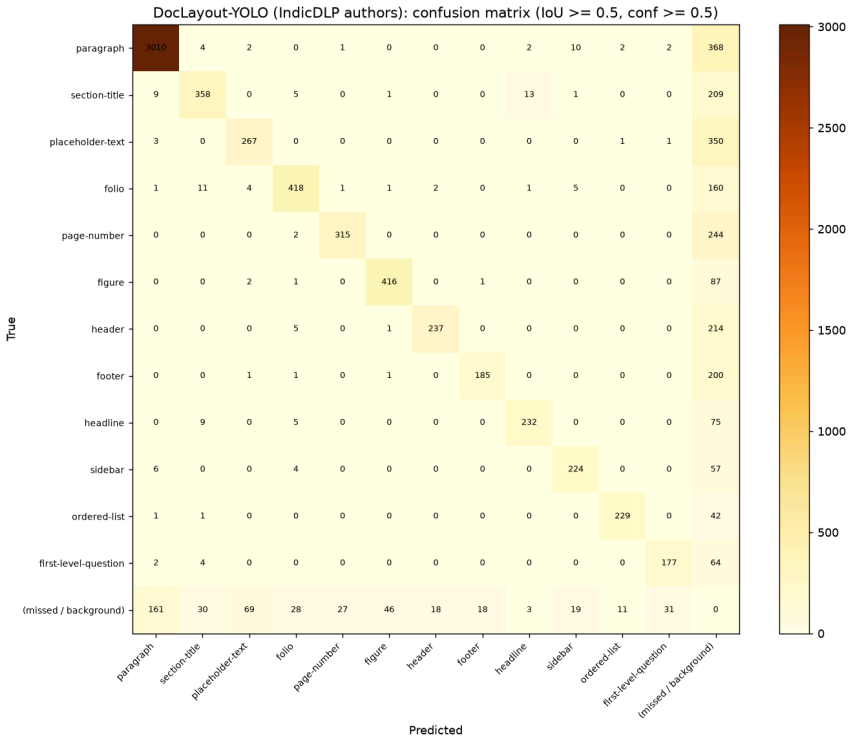


Fig. 14. Confusion matrix of DocLayout-YOLO for the most frequent labels (IoU ≥ 0.5, confidence ≥ 0.5). The last row counts missed regions; most errors are misses rather than wrong labels.

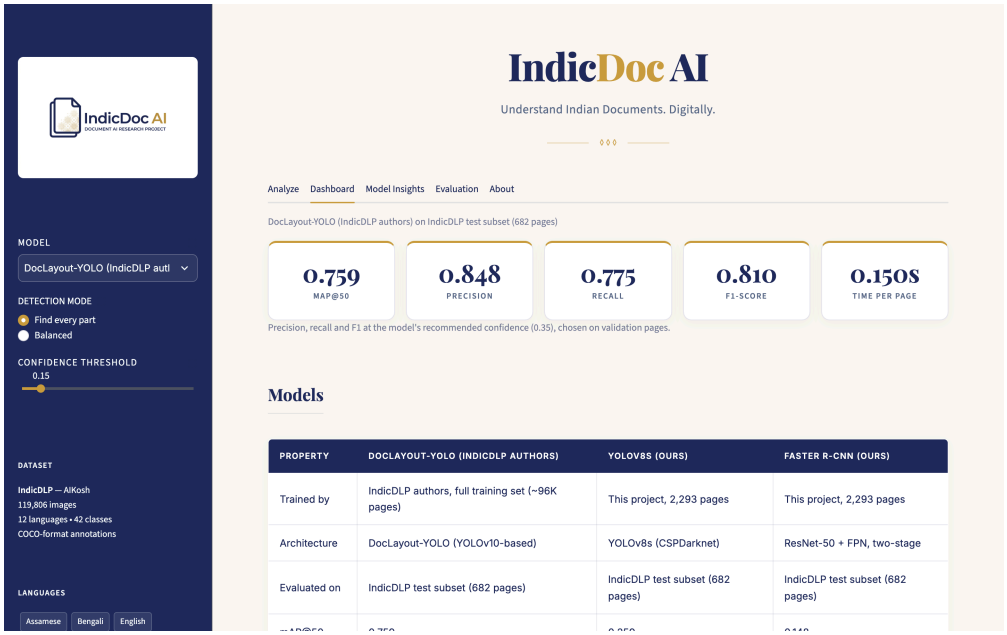


Fig. 15. Dashboard tab of the application showing the measured metrics of the three models.

15. Challenges and Limitations

1. **Data size and compute.** The full dataset is about 91 GB; a laptop GPU could train on 2,293 pages for about an hour per model. Our models stay far below the published one

(0.259 vs. 0.759 mAP@50), and each was trained once with one seed.

2. **Dense forms and rare labels.** Forms, with many tiny fields, score lowest; labels such as *quote* and *sub-unordered-list* score below 0.2 AP even for the best model.
3. **OCR quality.** Tesseract misreads small print and mis-orders vowel signs in some scripts; IndicDLP has no text ground truth, so character accuracy could not be measured. Scan damage in E2 is simulated.
4. **Engineering.** The initial code base had crashes, a wrong label list and plots not backed by data; every fix is logged in `docs/FINAL_AUDIT.md`. Timings were spoiled twice by jobs sharing the GPU and had to be re-measured.

16. Conclusion and Future Scope

Conclusion. We implemented a complete pipeline: data preparation from IndicDLP, two fine-tuned detectors, a controlled evaluation of three detectors, OCR with automatic script choice for 12 languages, accessible HTML / EPUB3 export, a web app, a REST API and an offline Docker image. The best model reaches 0.759 mAP@50 on 682 test pages, finds 85% of regions in "find every part" mode and runs at 0.15 s per page; the script is chosen correctly on 106/120 pages. The study shows that accuracy is fairly even across languages (CI 0.86) but not across document types (CI 0.55); that data size, not the script, decides how well rare labels work; and that the best model keeps 97% of its accuracy on damaged scans, with blur hurting most.

Objectives met. All four specific objectives (Section 4) were met. Our own detectors did not reach the published accuracy, which we report openly; the proposed YOLOv8s did improve on the Faster R-CNN baseline.

Future scope.

- *Model and data:* train longer on a cloud GPU with blur and noise augmentation; more labelled forms; a controlled unseen-script experiment.
- *OCR:* replace Tesseract with an Indic OCR model behind the same interface and build a small text ground-truth set to measure character accuracy.
- *Features:* exam-paper mode (questions, options and marks as structured items), multi-page books, Braille (.brf) and offline text-to-speech, and a review screen to correct OCR before export; table structure and question answering on detected tables [3], [4].
- *Real-world validation:* a pilot on state-board textbooks and government circulars with screen-reader users, measuring reading-order accuracy and time saved over manual conversion.

Expected Outcomes

TABLE XIV
EXPECTED OUTCOMES AND THEIR STATUS

Expected outcome	Status	Evidence
Development of a working Deep Learning model addressing a real-world problem	Achieved	Two fine-tuned detectors + released model in a working app and API (Sections 8–10, 14)
Performance evaluation of implemented model	Achieved	mAP, P / R / F1, IoU, speed, per-language / domain / label results, robustness, OCR script choice (Sections 12–13)
Improved understanding of research methodology and problem-solving skills	Achieved	Five research questions, held-out test set, thresholds chosen on validation only, audit log of fixes
Preparation of a research paper suitable for submission	Achieved	IEEE conference-format paper in `paper/`

Deliverables

TABLE XV
DELIVERABLES AND THEIR LOCATION IN THE REPOSITORY

Deliverable	Location	Contents
Dataset (CSV / image dataset)	`dataset/`	pages.csv (3,317 rows), boxes.csv (49,995 rows), class_counts.csv, language_split.csv, 12 sample pages
Project code with complete implementation	`ml/ app/ backend/ scripts/`	Training, evaluation, inference, OCR, accessible export, web app, REST API, Docker
GitHub repository with organized code and documentation	`README.md, docs/`	Dataset, architecture, methodology, evaluation, results, deployment and audit documents; tests
Presentation slides (Phase-1 and Phase-2)	`docs/presentation/`	Phase-1 literature survey (PDF); Phase-2 implementation (PDF + PPTX)
Final research paper	`paper/`	IEEE-format paper (PDF + LaTeX source)
Experiment report (this document)	`docs/report/`	Expt.-11 report; generated by `scripts/build_lab_report.py`

Evaluation Criteria

TABLE XVI
EVALUATION CRITERIA AND WHERE EACH IS ADDRESSED

Criterion	Marks	Where addressed
Relevance of problem to SDG	2	Section 1, Table I (SDG 4, 9, 10, 16 with targets)
Quality of literature survey and research gap identification	2	Sections 2–3; Phase-1 slides
Implementation of deep learning models	2	Sections 7–11 (data, three detectors, training)
Performance evaluation and Frontend development	2	Sections 12–14 (metrics, study, app, API)
Documentation and presentation quality	2	Repository docs, Phase-1 / Phase-2 slides, this report
Innovation and Research Paper	5	CI / RI measures, robustness test, script probe, accessible output; IEEE paper

References

[1] O. Nath, S. Kukkala, M. Khapra, and R. K. Sarvadevabhatla, "IndicDLP: A foundational dataset for multi-lingual and multi-domain document layout parsing," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, 2025, pp. 23–39, arXiv:2512.20236.

[2] B. V. Kasuba *et al.*, "PLATTER: A page-level handwritten text recognition system for Indic scripts," arXiv:2502.06172, 2025.

[3] A. Mondal *et al.*, "UniTabBank: A large scale multi-lingual, multi-layout, multi-type, multi-format dataset for table detection," in *Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV)*, 2026.

[4] V. Pal, E. Kanoulas, A. Yates, and M. de Rijke, "Table question answering for low-resourced Indic languages," arXiv:2410.03576, 2024.

[5] Y. Xu *et al.*, "LayoutXLM: Multimodal pre-training for multilingual visually-rich document understanding," arXiv:2104.08836, 2021.

[6] B. V. Kasuba, P. Chaudhuri, and G. Ramakrishnan, "DRISHTIKON: Visual grounding at multiple granularities in documents," arXiv:2506.21316, 2025.

[7] S. Bhattacharyya *et al.*, "Adapting vision-language models for Hindi OCR," IIIT Hyderabad, 2025.

[8] A. R. Dumada and S. G. Shah, "Incorporate structure and content for Devanagari table extraction," *Int. J. Sci. Adv. Technol. (IJSAT)*, vol. 17, no. 1, 2026.

[9] Z. Zhao, H. Kang, B. Wang, and C. He, "DocLayout-YOLO: Enhancing document layout analysis through diverse synthetic data and global-to-local adaptive perception," arXiv:2410.12628, 2024.

[10] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015, pp. 91–99.

[11] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: `https://github.com/ultralytics/ultralytics`

[12] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755.

[13] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.

[14] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[16] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 2117–2125.

[17] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 8024–8035.

[18] AI4Bharat, "IndicDLP dataset," Hugging Face, 2025. [Online]. Available: `https://huggingface.co/datasets/ai4bharat/indicdlp`

[19] O. Nath *et al.*, "IndicDLP model checkpoints," Zenodo, record 15881917, 2025, CC-BY 4.0. [Online]. Available: `https://zenodo.org/records/15881917`

[20] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, 2015.

[21] W3C, "EPUB Accessibility 1.1," W3C Recommendation, 2023. [Online]. Available: `https://www.w3.org/TR/epub-a11y-11/`

[22] Streamlit, FastAPI and Docker, official documentation. [Online]. Available: `https://docs.streamlit.io`, `https://fastapi.tiangolo.com`, `https://docs.docker.com`

[23] P. Jha, A. Soni, R. Patil, and M. Badure, "IndicDoc AI," source code, 2026. [Online]. Available: `https://github.com/va-ada/IndicDoc-AI`